KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

파이썬 기초 2 문자열, 리스트, for

글자를 자르고, 여러 값을 담아
보기 좋게 출력하기

```
word[0:2]          → 'py'
f"{pi:.2f}"        → 3.14
orders[1][0]       → 'latte'
len(words)         → 3
```

오늘의 순서

1

문자열과 출력

인덱스, 슬라이싱, sep, end, %, format(), f-string, 칸 수, 소수점, 콤마

4–18쪽

2

리스트와 튜플

[ ] 와 ( ), 자르기, 메서드, split과 join, 주문 데이터 orders, 꺼내 담기, 영수증 출력, for

19–43쪽

3쪽은 노트북 준비, 44–45쪽은 막힐 때와 한 장 요약이에요.

들어가기

12에서 이어서 해요

- **hello-uv 노트북, 커널 .venv** — 09 · 25–27쪽
- **변수, 타입, print()** — 12 · 1–2파트
- **계산과 +=, 비교** — 12 · 3–4파트

기억이 안 나면 그 쪽을 먼저 다시 봐요.

노트북 다시 열기

1. VS Code로 hello-uv 폴더 열기
2. 탐색기에서 `basics.ipynb` 열기
3. 오른쪽 위 커널이 .venv인지 보기
4. 셀에 적고 `Shift` + `Enter` 로 실행

이번 챕터의 코드는 모두 노트북 셀에 적어요.

1. 문자열과 출력

문자열은 따옴표로 감싸요

[1]
```python
a = 'kim'
b = "kim"
print(a == b)
print(len("hello"))
print("It's me")
```

```
True
5
It's me
```

`'kim'` `"kim"`

작은따옴표, 큰따옴표 모두 같은 문자열

a == b 는 True

`len("hello")`

글자 수를 세요

h e l l o 다섯 글자라서 5

`"It's me"`

글자 안에 ' 가 있으면

바깥을 " 로 감싸요

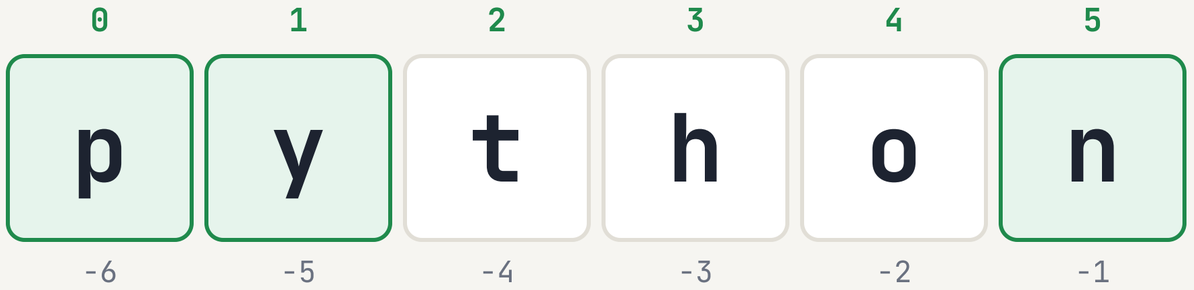
한 파일 안에서는 한 가지 따옴표로 맞춰 쓰면 읽기 쉬워요.

1. 문자열과 출력

[번호]로 글자 하나를 꺼내요

[2]
```python
word = "python"
print(word[0])
print(word[1])
print(word[-1])
```

```
p
y
n
```



위 숫자는 앞에서부터, 아래 숫자는 뒤에서부터 센 번호예요.

[5]
```python
word[6]
```

```
IndexError: string index out of range
```

0 번호는 1이 아니라 0부터 세요. 여섯 글자의 마지막 번호는 5예요.

word[-1]은 맨 뒤 글자예요. 길이를 몰라도 끝을 꺼낼 수 있어요.

1. 문자열과 출력

[시작:끝]으로 여러 글자를 잘라요

```
[3] word[0:2]
```
Python

```
'py'
```

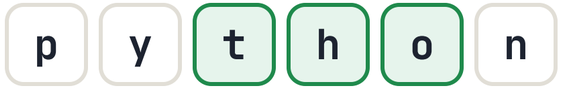
```
[4] print(word[2:5])
    print(word[:2])
    print(word[2:])
```
Python

```
tho
py
thon
```

word[0:2]
0번부터 2번 앞까지



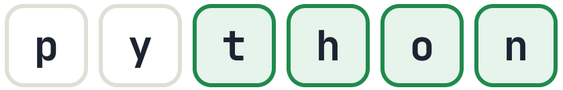
word[2:5]
2번부터 5번 앞까지



word[:2]
시작을 비우면 처음부터



word[2:]
끝을 비우면 끝까지



끝 번호의 글자는 들어가지 않아요. [0:2]는 0번, 1번 두 글자예요.

1. 문자열과 출력

print의 sep과 end로 모양을 바꿔요

[6]
```python
print("kim", "lee")
print("kim", "lee", sep="-")
print("2026", "11", "02", sep=".")
print("hello", end=" ")
print("world")
```

Python

```
kim lee
kim-lee
2026.11.02
hello world
```

sep 값 사이에 넣을 글자

`sep="-"  →  kim-lee`

기본은 빈칸 한 개

end 줄 끝에 붙일 글자

`end=" "  →  다음 print가 같은 줄에`

기본은 줄바꿈

print("설명", 값)이 빈칸으로 이어진 건 sep의 기본값이 빈칸이라서예요.

1. 문자열과 출력

\ 로 시작하는 특수 문자

[7]
```python
print("첫째 줄\n둘째 줄")
print("이름\tkim")
print("경로: C:\\Users")
```

```
첫째 줄
둘째 줄
이름	kim
경로: C:\Users
```

\n 줄바꿈
글자 사이에서 다음 줄로

\t 탭
칸을 띄워 줄 맞추기

**** 역슬래시 하나
\ 자체를 쓰고 싶을 때

Windows 키보드에서는 \ 가 ₩ 로 보여요. 같은 키예요.

1. 문자열과 출력

값을 끼워 넣는 방법은 세 가지예요

[8]
```python
name = "kim"
age = 25
print("%s 님은 %d살이에요" % (name, age))
print("{} 님은 {}살이에요".format(name, age))
print(f"{name} 님은 {age}살이에요")
```

```
kim 님은 25살이에요
kim 님은 25살이에요
kim 님은 25살이에요
```

% 포맷팅

`"%s" % name`

C 언어에서 온 가장 옛 방식

옛 코드와 책에서 자주 봐요

format()

`"{}".format(name)`

파이썬 3.0부터 (2008년)

중간 세대예요

f-string

`f"{name}"`

파이썬 3.6부터 (2016년)

요즘 표준. 새 코드는 이걸로 써요

세 줄 모두 결과가 같아요. 이 챕터는 % 와 f-string을 나란히 봐요.

1. 문자열과 출력

%s, %d 자리와 f-string의 { }

% 방식 · 옛 방식

[9]
```python
print("이름: %s" % name)
print("나이: %d살" % age)
```

```
이름: kim
나이: 25살
```

f-string · 요즘 방식

[10]
```python
print(f"이름: {name}")
print(f"나이: {age}살")
print(f"내년: {age + 1}살")
```

```
이름: kim
나이: 25살
내년: 26살
```

`"%s" % name`	→ `f"{name}"`	문자열 자리
`"%d" % age`	→ `f"{age}"`	정수 자리. f-string은 타입을 안 적어도 돼요
`"%s %d" % (name, age)`	→ `f"{name} {age}"`	여러 개면 % 는 괄호로 묶어 순서대로

f-string은 따옴표 앞에 f, 중괄호 안에 이름이나 계산식을 바로 적어요.

1. 문자열과 출력

format()은 {0} {1} {2} 번호로 값을 넣어요

[11]
```python
cake = "치즈케이크"
price = 5000
qty = 2
print("주문내역은 : {0}, {1}, {2}".format(cake, price, qty))
print("주문내역은 : {}, {}, {}".format(cake, price, qty))
print("{0} {2}개, 하나에 {1}원".format(cake, price, qty))
print("{0}, {0}, {0}".format(cake))
```

Python

```
주문내역은 : 치즈케이크, 5000, 2
주문내역은 : 치즈케이크, 5000, 2
치즈케이크 2개, 하나에 5000원
치즈케이크, 치즈케이크, 치즈케이크
```

{0} ← **cake**

0번째 값 → 치즈케이크

{1} ← **price**

1번째 값 → 5000

{2} ← **qty**

2번째 값 → 2

번호는 0부터 세요. 번호를 비우면 순서대로, 번호를 쓰면 순서를 바꾸거나 여러 번 쓸 수 있어요.

1. 문자열과 출력

같은 주문내역을 세 방식으로 적어 봐요

[12]
```python
print("주문내역은 : %s, %d, %d" % (cake, price, qty))
print("주문내역은 : {0}, {1}, {2}".format(cake, price, qty))
print(f"주문내역은 : {cake}, {price}, {qty}")
print(f"{cake} {qty}개, 하나에 {price}원")
print(f"{cake} {qty}개, 합계 {price * qty:,}원")
```

Python

```
주문내역은 : 치즈케이크, 5000, 2
주문내역은 : 치즈케이크, 5000, 2
주문내역은 : 치즈케이크, 5000, 2
치즈케이크 2개, 하나에 5000원
치즈케이크 2개, 합계 10,000원
```

% 방식

`%s, %d, %d`

자리 순서 = 값 순서

format()

`{0}, {1}, {2}`

번호로 값을 골라요

f-string

`{cake}, {price}, {qty}`

이름을 바로 적어요

f-string은 { } 안에 계산식과 모양도 넣어요. {price * qty:,} 는 합계에 콤마를 찍어요.

1. 문자열과 출력

%5d 와 {42:5d} 는 다섯 칸을 확보해요

% 방식 · 옛 방식

[13]
```python
print("[%5d]" % 42)
print("[%-5d]" % 42)
print("[%05d]" % 42)
print("[%5s]" % "kim")
```

```
[   42]
[42   ]
[00042]
[  kim]
```

f-string · 요즘 방식

[14]
```python
print(f"[{42:5d}]")
print(f"[{42:<5d}]")
print(f"[{42:05d}]")
print(f"[{'kim':>5}]")
print(f"[{'kim':5}]")
```

```
[   42]
[42   ]
[00042]
[  kim]
[kim  ]
```

`%5d`	`{42:5d}`	□ □ □ 4 2	다섯 칸, 오른쪽 맞춤
`%-5d`	`{42:<5d}`	4 2 □ □ □	왼쪽 맞춤. % 는 -, f-string은 <
`%05d`	`{42:05d}`	0 0 0 4 2	빈칸을 0으로
`%5s`	`{'kim':>5}`	□ □ k i m	글자 오른쪽 맞춤. f-string은 > 를 붙여요

f-string에서 글자는 기본이 왼쪽 맞춤이에요. {'kim':5}는 [kim]이 돼요.

1. 문자열과 출력

%0.2f 와 {pi:.2f} 는 소수점 두 자리예요

% 방식 · 옛 방식

[15]
```python
pi = 3.141592
print("%f" % pi)
print("%.2f" % pi)
print("%0.2f" % pi)
print("%.4f" % pi)
print("[%8.2f]" % pi)
```

```
3.141592
3.14
3.14
3.1416
[    3.14]
```

f-string · 요즘 방식

[16]
```python
print(f"{pi}")
print(f"{pi:.2f}")
print(f"{pi:0.2f}")
print(f"{pi:.4f}")
print(f"[{pi:8.2f}]")
```

```
3.141592
3.14
3.14
3.1416
[    3.14]
```

`%0.2f` `{pi:0.2f}`

`0` 전체 칸 수 `.2` 소수점 아래 자리 `f` 실수

0은 최소 칸 수라 영향이 없어서 0.2f 와 .2f 는 결과가 같아요.

1. 문자열과 출력

가격은 {price:,} 로 천 단위 콤마를 찍어요

[17]
```python
price = 1234567
print(f"{price:,}")
print(f"{price:,}원")
print(f"{1234567.891:,.2f}")
fee = 35000
print(f"[{fee:8,}]")
```

```
1,234,567
1,234,567원
1,234,567.89
[  35,000]
```

`{price:,}` → 1,234,567

콤마만

`{x:,.2f}` → 1,234,567.89

콤마와 소수점 두 자리

`{fee:8,}` → 35,000

여덟 칸 확보 + 콤마

[18]
```python
print("%,d" % price)
```

```
ValueError: unsupported format character ',' (0x2c) at index 1
```

% 방식에는 콤마 옵션이 없어요. 콤마가 필요하면 f-string을 써요.

콤마는 보여 주는 모양만 바꿔요. price 값은 그대로 1234567이에요.

1. 문자열과 출력

format()도 콜론 뒤 모양은 f-string과 같아요

[19]
```python
price = 5000
print("[{0:5d}]".format(qty))
print("{0:.2f}".format(3.141592))
print("{0:,}원".format(1234567))
print("{0}: {1:,}원 x {2}개".format(cake, price, qty))
```

```
[    2]
3.14
1,234,567원
치즈케이크: 5,000원 x 2개
```

`{0:5d}`	→	`{qty:5d}`	다섯 칸 확보
`{0:.2f}`	→	`{pi:.2f}`	소수점 두 자리
`{1:,}`	→	`{price:,}`	천 단위 콤마

f-string은 이름을, format()은 번호를 콜론 앞에 적어요. 콜론 뒤는 똑같아요.

1. 문자열과 출력

칸을 맞추면 표처럼 보여요

% 방식 · 옛 방식

[20]
```python
print("%-6s %5d" % ("kim", 90))
print("%-6s %5d" % ("lee", 100))
print("%-6s %5d" % ("park", 7))
print("할인 %d%%" % 30)
```

```
kim        90
lee       100
park        7
할인 30%
```

f-string · 요즘 방식

[21]
```python
print(f"{'kim':<6} {90:5d}")
print(f"{'lee':<6} {100:5d}")
print(f"{'park':<6} {7:5d}")
print(f"할인 {30}%")
```

```
kim        90
lee       100
park        7
할인 30%
```

`%-6s`	→ `{:<6}`	여섯 칸 왼쪽 맞춤 (이름)
`%5d`	→ `{:5d}`	다섯 칸 오른쪽 맞춤 (점수)
`%%`	→ `%`	% 글자 그대로. % 방식만 두 번 적어요

성적표, 영수증처럼 줄을 맞춰 보여 줄 때 칸 수를 써요.

1. 문자열과 출력

자리와 값이 안 맞으면 오류가 나요

[22]
```python
"%d살" % "kim"
```
```
TypeError: %d format: a real number is required, not str
```

% 방식
%d 자리에 글자를 넣었어요

[23]
```python
"%s 님은 %d살" % name
```
```
TypeError: not enough arguments for format string
```

% 방식
자리는 둘, 값은 하나예요

[24]
```python
"%s" % (name, age)
```
```
TypeError: not all arguments converted during string formatting
```

% 방식
자리는 하나, 값은 둘이에요

[25]
```python
f"{name:d}"
```
```
ValueError: Unknown format code 'd' for object of type 'str'
```

f-string
글자에 정수 모양 d를 줬어요

% 는 자리 수와 값 수를, f-string은 콜론 뒤 모양과 타입을 먼저 봐요.

2. 리스트와 튜플

리스트는 값 여러 개를 [ ] 에 담아요

[1]
```python
prices = [4500, 5000, 6500]
print(prices)
print(len(prices))
print(type(prices))
```

```
[4500, 5000, 6500]
3
<class 'list'>
```

prices

0	1	2
4500	5000	6500

`[ ]` 대괄호로 감싸고 쉼표로 나눠요

`len(prices)` 담긴 개수예요. 3

`<class 'list'>` 타입 이름은 list예요

변수 하나에 값 하나였다면, 리스트는 변수 하나에 값 여러 개예요.

2. 리스트와 튜플

꺼내는 법은 문자열과 같아요

[2]
```python
menus = ["coffee", "latte", "cake"]
print(menus[0])
print(menus[-1])
print(menus[0:2])
```

```
coffee
cake
['coffee', 'latte']
```



`menus[0]` → `'coffee'`

0번, 맨 앞

`menus[-1]` → `'cake'`

뒤에서 첫째

`menus[0:2]` → `['coffee', 'latte']`

0번부터 2번 앞까지. 결과도 리스트

1파트의 word[0], word[0:2]와 똑같아요. 번호는 0부터 세요.

2. 리스트와 튜플

리스트는 값을 바꾸고 더할 수 있어요

[3]
```python
prices[0] = 4800
prices.append(3000)
print(prices)
print(sum(prices))
print(max(prices))
print(min(prices))
```

```
[4800, 5000, 6500, 3000]
19300
6500
3000
```

처음



`prices[0] = 4800`



`prices.append(3000)`



`sum()` `max()` `min()` 합계, 가장 큰 값, 가장 작은 값

append는 맨 뒤에 하나를 더해요. 담긴 개수가 3에서 4가 돼요.

넣고 빼는 메서드: append, insert, remove, pop

[4]
```python
cart = ["coffee", "latte"]
cart.append("cake")
cart.insert(0, "tea")
cart.remove("latte")
print(cart)
last = cart.pop()
first = cart.pop(0)
print(last, first)
print(cart)
```

```
['tea', 'coffee', 'cake']
cake tea
['coffee']
```

[5]
```python
cart.remove("juice")
```

```
ValueError: list.remove(x): x not in list
```

cart.append("cake") 맨 뒤에 하나 더하기



cart.insert(0, "tea") 0번 자리에 끼워 넣기. 나머지는 뒤로 밀려요



cart.remove("latte") 값을 찾아 지우기. 돌려주는 값은 없어요



cart.pop() 맨 뒤를 빼서 돌려줘요



cart.pop(0) 0번을 빼서 돌려줘요

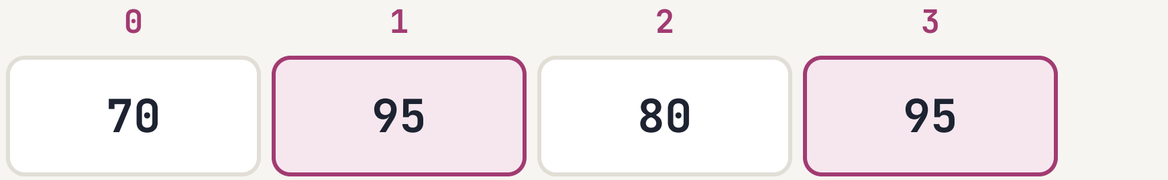


pop은 뺀 값을 돌려줘서 변수에 받을 수 있어요. 리스트는 하나 줄어요.

찾고 세고 정렬하는 메서드: count, index, sort

[6]
```python
scores = [70, 95, 80, 95]
print(scores.count(95))
print(scores.index(80))
print(95 in scores)
scores.sort()
print(scores)
scores.sort(reverse=True)
print(scores)
```

```
2
2
True
[70, 80, 95, 95]
[95, 95, 80, 70]
```



`scores.count(95)`	**2** 95가 몇 개인지
`scores.index(80)`	**2** 80이 몇 번 자리인지
`95 in scores`	**True** 들어 있는지 True, False로
`scores.sort()`	**[70, 80, 95, 95]** 작은 수부터 정렬. 리스트가 바뀌어요
`scores.sort(reverse=True)`	**[95, 95, 80, 70]** 큰 수부터 정렬

index의 답도 0부터 센 번호예요. 80은 0, 1, 2번째라서 2예요.

2. 리스트와 튜플

리스트에 쓰는 메서드와 함수 한눈에

넣기

`append(값)`
맨 뒤에 더하기

`insert(번호, 값)`
그 자리에 끼우기

빼기

`remove(값)`
값을 찾아 지우기

`pop()`
맨 뒤를 꺼내기

찾기

`count(값)`
몇 개인지

`index(값)`
몇 번 자리인지

정렬

`sort()`
작은 수부터

`sort(reverse=True)`
큰 수부터

점을 찍는 메서드와 괄호로 감싸는 함수

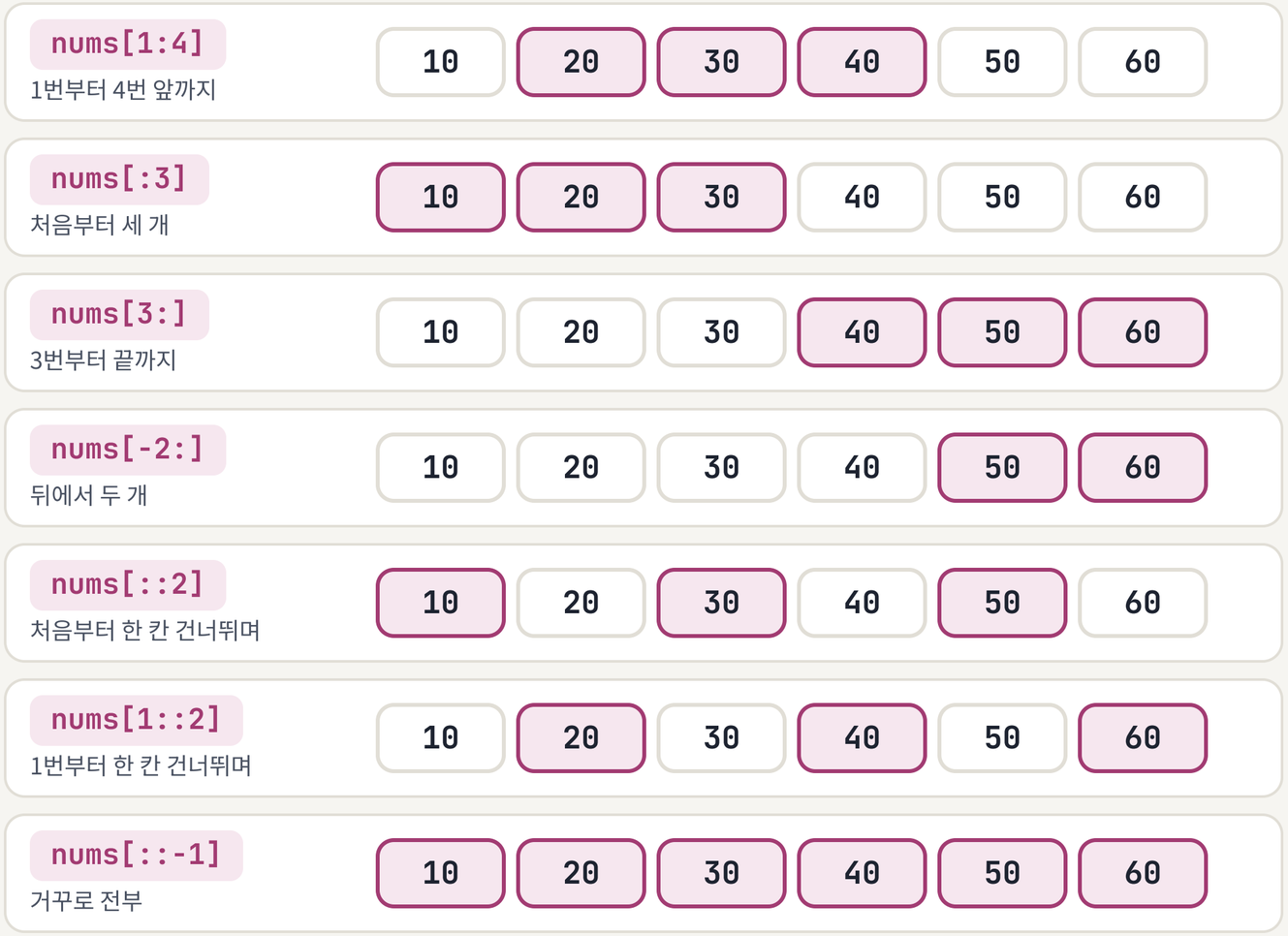
`cart.append("cake")`
메서드. 리스트 이름 뒤에 점

`len(cart)` `sum(prices)` `max(prices)` `min(prices)`
함수. 괄호 안에 리스트

[시작:끝:간격]으로 원하는 만큼 잘라요

[7]
```python
nums = [10, 20, 30, 40, 50, 60]
print(nums[1:4])
print(nums[:3])
print(nums[3:])
print(nums[-2:])
print(nums[::2])
print(nums[1::2])
print(nums[::-1])
```

```
[20, 30, 40]
[10, 20, 30]
[40, 50, 60]
[50, 60]
[10, 30, 50]
[20, 40, 60]
[60, 50, 40, 30, 20, 10]
```



세 번째 숫자는 간격이에요. 2면 한 칸씩 건너뛰고, -1이면 거꾸로 가요.

안쪽 리스트와 글자도 번호로 꺼내요

[8]
```python
table = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
print(table[1])
print(table[1][2])
print(table[-1][-1])
print(table[0][:2])
print(menus[0][0])
print(menus[1][-3:])
```

```
[4, 5, 6]
6
9
[1, 2]
c
tte
```

	0	1	2
table[0]	1	2	3
table[1]	4	5	6
table[2]	7	8	9

`table[1][2]` 1번 줄의 2번 칸 → 6

`table[-1][-1]` 마지막 줄의 마지막 칸 → 9

`table[0][:2]` 0번 줄을 앞에서 두 개 → [1, 2]

`menus[0][0]` `menus[1][-3:]` 글자도 꺼내고 잘라요 → c, tte

2. 리스트와 튜플

리스트를 잇고, 뒤집고, 복사하고, 비워요

[9]
```python
a = [3, 1, 2]
b = [5, 4]
print(a + b, a * 2)
a.extend(b)
print(a)
print(sorted(a), a)
a.reverse()
print(a)
c = a.copy()
c.clear()
print(a, c)
```

```
[3, 1, 2, 5, 4] [3, 1, 2, 3, 1, 2]
[3, 1, 2, 5, 4]
[1, 2, 3, 4, 5] [3, 1, 2, 5, 4]
[4, 5, 2, 1, 3]
[4, 5, 2, 1, 3] []
```

`a + b` `a * 2`

두 리스트를 이은 새 리스트, 두 번 반복한 새 리스트

`a.extend(b)`

a 뒤에 b를 이어 붙여요. a가 바뀌어요

`sorted(a)`

정렬한 새 리스트를 돌려줘요. a는 그대로

`a.reverse()`

a를 거꾸로 뒤집어요. 정렬이 아니라 순서만 뒤집기

`a.copy()` `c.clear()`

복사본을 만들고, 복사본만 비워요. a는 그대로

split 으로 글자를 리스트로, join 으로 리스트를 글자로

[10]
```python
text = "coffee,latte,cake"
words = text.split(",")
print(words)
print(len(words))
print(" / ".join(words))
print("hello world".split())
print(list("abc"))
```

```
['coffee', 'latte', 'cake']
3
coffee / latte / cake
['hello', 'world']
['a', 'b', 'c']
```

`"coffee,latte,cake"`

`.split(",")` 쉼표에서 잘라

`['coffee', 'latte', 'cake']`

`['coffee', 'latte', 'cake']`

`" / ".join(words)` 사이에 / 를 넣어 이어서

`"coffee / latte / cake"`

split()처럼 괄호를 비우면 빈칸에서 잘라요.

list("abc")는 글자를 한 글자씩 나눠 리스트로 만들어요.

2. 리스트와 튜플

튜플은 ( ) 에 담고 바꿀 수 없어요

[11]
```python
order = ("coffee", 4500, 2)
print(order[0])
print(order[1] * order[2])
print(type(order))
```
Python

```
coffee
9000
<class 'tuple'>
```

[ ] 리스트

값을 바꾸고 더할 수 있어요. 장바구니처럼 늘어나는 것

( ) 튜플

만든 뒤 못 바꿔요. 주문 한 건처럼 한 묶음인 것

[12]
```python
order[1] = 5000
```
Python

```
TypeError: 'tuple' object does not support item assignment
```

1파트의 "%s %d" % (name, age)에서 괄호로 묶은 것도 튜플이었어요.

튜플은 count, index, 꺼내 담기, 바꾸기

[13]
```python
date = (2026, 11, 2)
print(date.count(2))
print(date.index(11))
year, month, day = date
print(year, month, day)
a = "kim"
b = "lee"
a, b = b, a
print(a, b)
print(list(date))
print(tuple(menus))
```

```
1
1
2026 11 2
lee kim
[2026, 11, 2]
('coffee', 'latte', 'cake')
```

`date.count(2)` `date.index(11)`

튜플의 메서드는 둘뿐이에요. 바꾸는 메서드가 없어요

`year, month, day = date`

튜플 값을 변수 여러 개에 한 번에 꺼내 담아요

`a, b = b, a`

두 변수의 값을 맞바꿔요. lee kim

`list(date)` `tuple(menus)`

튜플을 리스트로, 리스트를 튜플로 바꿔요

날짜, 좌표, 주문 한 건처럼 바뀌지 않는 한 묶음은 튜플에 담아요.

2. 리스트와 튜플

튜플도 꺼내고, 자르고, 이어요

[14]

```python
t = (10, 20, 30, 40)
print(t[1:3])
print(t[::-1])
print(t + (50,))
print(len(t), sum(t), max(t))
print(30 in t)
info = ("kim", (2026, 11, 2))
print(info[1][0])
```

Python

```
(20, 30)
(40, 30, 20, 10)
(10, 20, 30, 40, 50)
4 100 40
True
2026
```

`t[1:3]` `t[::-1]`

자르기는 리스트와 같아요. 결과는 튜플

`t + (50,)`

이어 붙인 새 튜플. 값 하나짜리 튜플은 뒤에 쉼표

`len(t)` `sum(t)` `max(t)` `30 in t`

함수와 in도 그대로 써요

`info[1][0]`

튜플 안의 튜플도 번호 두 번 → 2026

바꾸는 것만 안 되고, 꺼내고 자르는 건 리스트와 똑같아요.

주문 여러 건은 튜플을 리스트에 담아요

[15]
```python
orders = [("coffee", 4500, 2), ("latte", 5000, 1), ("cake", 6500, 3)]
print(len(orders))
print(orders[1])
print(orders[1][0])
print(orders[2][1] * orders[2][2])
print(orders[0:2])
```

```
3
('latte', 5000, 1)
latte
19500
[('coffee', 4500, 2), ('latte', 5000, 1)]
```

앞 번호는 몇째 주문,
뒤 번호는 몇째 값

	[0] 메뉴	[1] 가격	[2] 수량
orders[0]	coffee	4500	2
orders[1]	latte	5000	1
orders[2]	cake	6500	3

orders[1][0]
1번 주문의 0번 값 → 'latte'

orders[2][1] * orders[2][2]
cake 가격 × 수량 → 19500

튜플의 값을 변수 여러 개에 한 번에 꺼내 담아요

[16]
```python
menu, price, qty = orders[2]
print(menu, price, qty)
print(f"{menu} {qty}개, {price * qty:,}원")
```

```
cake 6500 3
cake 3개, 19,500원
```

```
orders[2] = ("cake", 6500, 3)
```

menu	price	qty
'cake'	6500	3

왼쪽 이름 수와 튜플 값 수가 같아야 해요. 순서대로 하나씩 들어가요.

2. 리스트와 튜플

f-string으로 주문 영수증을 찍어요

[17]
```python
print(f"{orders[0][0]:<7}{orders[0][1]:>6,}원 {orders[0][2]:>2}개")
print(f"{orders[1][0]:<7}{orders[1][1]:>6,}원 {orders[1][2]:>2}개")
print(f"{orders[2][0]:<7}{orders[2][1]:>6,}원 {orders[2][2]:>2}개")
total = orders[0][1] * orders[0][2]
total += orders[1][1] * orders[1][2]
total += orders[2][1] * orders[2][2]
print(f"합계 {total:,}원")
```

```
coffee  4,500원  2개
latte   5,000원  1개
cake    6,500원  3개
합 계 33,500원
```

`{orders[0][0]:<7}`
메뉴를 일곱 칸 왼쪽 맞춤

`{orders[0][1]:>6,}`
가격을 여섯 칸, 콤마

`{orders[0][2]:>2}`
수량을 두 칸 오른쪽 맞춤

주문 한 건에 한 줄씩 적었어요. 줄이 같은 모양이라 칸이 맞아요.

머리글의 한글은 화면에서 두 칸을 차지해요

[18]
```python
print(f"{'메뉴명':<10}{'단가':>8}{'수량':>5}{'금액':>10}")
print(f"{'menu':<10}{'price':>8}{'qty':>5}{'amount':>10}")
print(f"{orders[0][0]:<10}{orders[0][1]:>8,}{orders[0][2]:>5}{orders[0][1] * orders[0][2]:>10,}")
```

```
메 뉴 명              단 가    수 량         금 액
menu          price   qty     amount
coffee        4,500     2      9,000
```

< > ^

왼쪽, 오른쪽, 가운데 맞춤

```
{'메뉴명':<10} {'단가':>8} {'qty':^5}
```

^ 는 남는 칸을 양쪽에 나눠요

한글은 두 칸

파이썬은 한 글자로 세는데 화면에선 두 칸이에요

```
'메뉴명'은 세 글자지만 화면에선 여섯 칸
```

그래서 머리글만 오른쪽으로 밀려요

칸 수 줄이기

한글 글자 수만큼 칸 수를 빼요

```
{'메뉴명':<7} {'단가':>6} {'수량':>3} {'금액':>8}
```

글꼴에 따라 조금 어긋나면 머리글을 영어로 써요

영어 머리글은 칸이 맞고, 한글 머리글만 밀렸어요.

머리글, 줄, 합계를 붙여 영수증을 완성해요

[19]
```python
print("=" * 33)
print(f"{'메뉴명':<7}{'단가':>6}{'수량':>3}{'금액':>8}")
print("-" * 33)
print(f"{orders[0][0]:<10}{orders[0][1]:>8,}{orders[0][2]:>5}{orders[0][1] * orders[0][2]:>10,}")
print(f"{orders[1][0]:<10}{orders[1][1]:>8,}{orders[1][2]:>5}{orders[1][1] * orders[1][2]:>10,}")
print(f"{orders[2][0]:<10}{orders[2][1]:>8,}{orders[2][2]:>5}{orders[2][1] * orders[2][2]:>10,}")
print("-" * 33)
print(f"{'합계':<6}{total:>25,}")
print("=" * 33)
```

Python

```
=================================
메뉴명         단가  수량       금액
---------------------------------
coffee       4,500    2     9,000
latte        5,000    1     5,000
cake         6,500    3    19,500
---------------------------------
합계                        33,500
=================================
```

for 는 리스트에서 하나씩 꺼내 같은 일을 되풀이해요

[20]
```python
for menu in menus:
    print(menu)
```

```
coffee
latte
cake
```

`for menu in menus:`

menus에서 하나씩 꺼내 menu에 넣어요. 끝에 콜론

`    print(menu)`

네 칸 들여 쓴 줄이 되풀이할 일이에요

1번째 `menu = 'coffee'` → `print(menu)` → coffee

2번째 `menu = 'latte'` → `print(menu)` → latte

3번째 `menu = 'cake'` → `print(menu)` → cake

값이 세 개라서 세 번 되풀이하고 끝나요.

튜플을 꺼내 담으면서 되풀이해요

[21]
```python
for menu, price, qty in orders:
    print(menu, price, qty)
```

```
coffee 4500 2
latte 5000 1
cake 6500 3
```

for 없이 적으면

```python
menu, price, qty = orders[0]
print(menu, price, qty)
menu, price, qty = orders[1]
print(menu, price, qty)
menu, price, qty = orders[2]
print(menu, price, qty)
```

한 번 돌 때마다 주문 튜플 하나가 세 이름에 나눠 들어가요

	menu	price	qty
1번째 · orders[0]	coffee	4500	2
2번째 · orders[1]	latte	5000	1
3번째 · orders[2]	cake	6500	3

되풀이하면서 합계를 쌓아요

[22]
```python
total = 0
for menu, price, qty in orders:
    total += price * qty
    print(menu, total)
print("합계:", total)
```

```
coffee 9000
latte 14000
cake 33500
합계: 33500
```

	price * qty	total
시작		0
1번째 coffee	4500 * 2 = 9000	9000
2번째 latte	5000 * 1 = 5000	14000
3번째 cake	6500 * 3 = 19500	33500

total = 0 은 for 앞에서 한 번만, total += 는 안쪽에서 매번 실행돼요.

for 로 영수증 한 장을 완성해요

[23]
```python
print("=" * 33)
print(f"{'KANT 카페':^31}")
print(f"{'2026-11-02  14:30':^33}")
print("=" * 33)
print(f"{'메뉴명':<7}{'단가':>6}{'수량':>3}{'금액':>8}")
print("-" * 33)
total = 0
count = 0
for menu, price, qty in orders:
    print(f"{menu:<10}{price:>8,}{qty:>5}{price * qty:>10,}")
    total += price * qty
    count += qty
print("-" * 33)
print(f"{'수량':<6}{count:>15}")
print(f"{'합계':<6}{total:>25,}")
print("=" * 33)
print(f"{'감사합니다':^28}")
```
Python

실행 결과

```
=================================
            KANT 카페
        2026-11-02  14:30
=================================
메뉴명          단가  수량        금액
---------------------------------
coffee       4,500     2     9,000
latte        5,000     1     5,000
cake         6,500     3    19,500
---------------------------------
수량                       6
합계                          33,500
=================================
          감사합니다
```

한 셀을 실행하면 영수증 한 장이 나와요. 주문이 늘어도 코드는 그대로예요.

for 는 콜론과 들여쓰기를 꼭 지켜요

[24]
```python
for menu in menus
    print(menu)
```

```
SyntaxError: expected ':'
```

for 줄 끝에 콜론을 빠뜨렸어요

[25]
```python
for menu in menus:
print(menu)
```

```
IndentationError: expected an indented block after 'for' statement on line 1
```

되풀이할 줄을 들여 쓰지 않았어요

Tab VS Code에서 콜론 뒤 Enter를 치면 네 칸이 자동으로 들어가요. 직접 넣을 땐 Tab 한 번이에요.

2. 리스트와 튜플

리스트에는 여러 모양의 값을 담아요

[26]
```python
names = ["kim", "lee", "park"]
scores = [90, 100, 7]
mixed = ["coffee", 4500, True]
carts = [["coffee", "cake"], ["latte"]]
empty = []
print(scores[1])
print(mixed)
print(carts[0][1])
print(len(empty))
```

```
100
['coffee', 4500, True]
cake
0
```

`["kim", "lee"]`
글자 리스트

`[90, 100, 7]`
숫자 리스트. sum, max를 쓸 수 있어요

`["coffee", 4500, True]`
타입을 섞어도 돼요

`[["coffee", "cake"], ["latte"]]`
리스트 안의 리스트. 손님별 장바구니

`[("coffee", 4500, 2)]`
튜플의 리스트. 앞 장의 orders

`[]`
빈 리스트. 나중에 append로 채워요

carts[0][1]처럼 안쪽 리스트도 번호를 두 번 써서 꺼내요.

없는 번호를 꺼내면 IndexError가 나요

[27]
```python
menus[3]
```
```
IndexError: list index out of range
```

menus는 0, 1, 2번까지

[28]
```python
orders[3]
```
```
IndexError: list index out of range
```

주문은 세 건, 마지막은 2번

[29]
```python
orders[0][3]
```
```
IndexError: tuple index out of range
```

튜플 안은 0, 1, 2번까지

[12]
```python
order[1] = 5000
```
```
TypeError: 'tuple' object does not support item assignment
```

튜플은 바꿀 수 없어요

마지막 번호는 len() - 1이에요. 메시지에 list인지 tuple인지 나와요.

정리

막히면 오류의 마지막 줄을 봐요

1	`IndexError: list index out of range` 없는 번호를 꺼냈어요	마지막은 len() - 1
2	`TypeError: not enough arguments for format string` % 자리 수와 값 수가 달라요	f-string으로
3	`ValueError: list.remove(x): x not in list` 리스트에 없는 값을 지우려 했어요	in 으로 먼저 확인
4	`TypeError: 'tuple' object does not support item assignment` 튜플은 담은 뒤 바꿀 수 없어요	바꿀 값은 리스트로
5	`SyntaxError: expected ':'` for 줄 끝에 콜론이 없어요	for x in xs:
6	`IndentationError: expected an indented block after 'for' statement on line 1` for 안쪽 줄을 들여 쓰지 않았어요	네 칸 들여쓰기

오류가 나면 위쪽 Traceback보다 맨 아래 줄부터 읽어요.

한 장 요약

`word[0:2]`

앞에서 두 글자 자르기

`f"{name} {age}살"`

값을 끼워 넣기

`f"{price:,}원"`

콤마 찍어 출력

`f"{pi:.2f}"`

소수점 둘째 자리까지

`prices.append(5000)`

리스트 끝에 하나 더하기

`orders[1][0]`

리스트 안 튜플에서 꺼내기

`a, b, c = order`

튜플 값을 나눠 담기

`for x in xs:`

하나씩 꺼내 되풀이

번호는 0부터 세요. `[시작:끝]`에서 끝 번호 자리는 빠져요

리스트 `[ ]`는 바꿀 수 있고, 튜플 `( )`는 바꿀 수 없어요